
html · css · javascript

100 days of web development

One assignment a day, building a gaming community website from the first tag to launch day.

Level 1
Structure

Level 2
Style

Level 3
Behaviour

Level 4
The platform

Most of it works on a phone. The days where a laptop makes a real difference are marked, and listed on one page near the front.

Contents

Level 1	Structure	Days 1 to 14
Boss week 1	The whole Lobby in HTML	days 11 to 14
Level 2	Style	Days 15 to 37
Boss week 2	Lobby gets its look	days 32 to 37
Level 3	Behaviour	Days 38 to 72
Boss week 3	Match night: a live scoreboard	days 57 to 63
Level 4	The platform	Days 73 to 100
Boss week 4	The game library	days 73 to 79
Boss week 5	Tournaments and brackets	days 80 to 86
Boss week 6	Players, clans and the real leaderboard	days 87 to 93
Boss week 7	Launch week	days 94 to 100

How this guide works

This guide is for someone who has already written a bit of HTML, has seen some CSS, and maybe copied a few lines of JavaScript without fully understanding them. You do not need to be good yet. You just need to have started.

For 100 days you build one website: **Lobby**, a home for a gaming community. It has tournaments people can register for, a leaderboard worked out from real match results, player and clan profiles, a game library you can search and filter, and a match night scoreboard you can run on one phone while everyone plays. Lobby is a placeholder name. Call yours whatever your crew is called.

Every day has the same parts:

- **Learn** is a short explanation with an example. Read it, then type the example yourself.
- **Your task** is the day's assignment. It always adds something real to your site.
- **Done when** is a checklist. The day is finished when every box is true, not when you are tired of it.
- **If you are stuck** gives one hint, on the days where people usually get stuck.
- **Push further** is optional, for days when you have energy left.

Most days take 45 to 90 minutes. Some take less. A few will take longer, and that is fine.

Boss weeks

Seven times in the 100 days, the daily tasks stop and a boss week starts. A boss week is one bigger project spread over four to seven days, with a plan for each day. From day 73 to day 100 it is boss weeks only, because the features you are building by then are too big to finish in an evening.

Every boss week has a **Must have** list and a **Nice to have** list. Finish every must-have item before you touch a nice-to-have. The must-have list is designed to be doable. If you finish it and nothing else, you have passed the week.

If you fall behind

Take two days over one task if you need to. Skip a day's reading if you already know it. The one thing not to skip is the task itself, because every day builds on the site you had yesterday.

If you miss a whole week, do not try to catch up by rushing. Carry on from where you stopped. The dates on the tracker at the back are yours to move.

Looking things up

Nobody memorises all of this. When you need the details of an HTML element, a CSS property or a JavaScript method, look it up on **MDN** (developer.mozilla.org). It is free, accurate and works well on a phone.

When something breaks, read the error message before you search for anything. It usually tells you the problem and the line number.

AI chat tools are good at explaining code you do not understand. Use them for that. Do not ask one to do your task for you. If you cannot explain a line of your own site, you have not learned it yet.

Keep a backup

Once a week, copy your whole project folder somewhere safe: another folder, Google Drive, or GitHub from day 37 onwards. Phones get lost, stolen and reset.

Setting up on a phone

You can do almost all of this guide on a phone. What you need is a code editor app that can hold a folder of files and show you a preview of your site.

Android

Install **Acode** from the Play Store. It is free, it opens a real folder on your phone, and its run button shows your site in a preview that works like a small web server. **Spck Editor** is a good alternative with a similar setup.

1. Make a folder called `lobby` in your phone's Documents folder.
2. In Acode, open that folder.
3. Create a file called `index.html`.
4. Use the run or preview button to see your page.

iPhone

Look for an editor that can open a folder of files and preview HTML. **Textastic** (paid) is a long-standing option, and **Koder** is another. App features and prices change, so check the current App Store listing for a preview feature before you pay. If you cannot find one that suits you, you can run the whole project in **Replit** from your phone browser.

A console on your phone

Phone browsers do not show you the console, which is where JavaScript errors and your `console.log` messages appear. You will need one from day 38. Add these two lines just before your own script while you are building, and a small console button appears on your page:

```
index.html

<script src="https://cdn.jsdelivr.net/npm/eruda"></script>
<script>eruda.init();</script>
```

It needs internet to load. Take it out before you launch.

Make typing easier

Code is full of brackets and symbols. Two things help a lot:

- Turn your phone sideways when you edit. You get a wider line and a bigger keyboard.
- A cheap Bluetooth keyboard makes a bigger difference than any app. If you are going to do all 100 days on a phone, it is worth buying one.

Data and internet

Almost everything works offline. You need internet for the days that load fonts from Google (day 17), embed YouTube videos (day 10), use Eruda, talk to GitHub (days 37 and 67) and test on your live site. Everything else

can be done with your data switched off.

What the day labels mean

Each day is labelled with what you need.

- **Phone** means it works in your phone editor.
- **Phone + server** means it works on a phone, but only through your editor's preview or on your live GitHub Pages site. From day 64 your site loads data files with `fetch`, and browsers refuse to do that for a file opened straight from storage. Opening the file directly will look broken even when your code is right.
- **Laptop recommended** means it is possible on a phone, but much easier on a laptop. The next page lists those days.

Setting up on a laptop

A laptop with 4 GB of memory is enough for everything in this guide. It does not need to be new.

1. Install **Google Chrome** or **Microsoft Edge**. Their developer tools are what this guide describes.
2. Install **Visual Studio Code** from code.visualstudio.com. It is free.
3. In VS Code, open the Extensions panel and install **Live Server**. It serves your folder so pages reload when you save, and it makes `fetch` work.
4. Open your `lobby` folder with File, then Open Folder. Right-click `index.html` and choose Open with Live Server.
5. Press F12 in the browser to open the developer tools. The Console tab shows errors. The Elements tab shows your HTML and the CSS applied to it. Ctrl+Shift+M switches on the device toolbar, which shows your site at phone and tablet widths.

Git and GitHub

From day 37 your site lives on GitHub Pages. On a phone you upload files through the GitHub website. On a laptop it is quicker to use Git.

1. Install Git from git-scm.com, and make a free account at github.com.
2. Tell Git who you are, once:

```
terminal
```

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"
```

1. In your project folder, run `git init` once. After that, each time you finish a day:

```
terminal
```

```
git add .  
git commit -m "Day 23: homepage grid layout"  
git push
```

A commit message per day also gives you a diary of your 100 days.

Moving from phone to laptop

Your project is only a folder of files, so moving is easy. Zip the folder on your phone and send it to your laptop, or once you are on GitHub, download it from there. There is nothing to install and nothing to convert.

When you need a laptop

Nothing in this guide is impossible on a phone. These are the days where a laptop saves you a lot of time, usually because the job needs a real keyboard, desktop-width screens, or the full developer tools. Each of those days also says what to do if you only have a phone.

Day	Topic
Day 9	Make it work for everyone
Day 28	Mobile first, then bigger screens
Day 56	Debugging like you mean it
Day 71	JavaScript that works for everyone
Day 80 to 86	Boss week 5: Tournaments and brackets
Day 94 to 100	Boss week 7: Launch week

The reasons come down to four things. Checking that someone can use your site without a mouse means pressing Tab through it, which needs a keyboard, though a Bluetooth one connected to your phone works too. A phone only shows phone widths, while a laptop's device toolbar shows every size in seconds. Pausing JavaScript line by line to watch what it does needs desktop developer tools. And boss weeks 5 and 7 involve a lot of code across several files at once. You can do them on a phone; you will just be slower and more frustrated.

What you are building

By day 100, Lobby has these pages. You start them in the first two weeks and keep improving them until launch.

- **Home.** What the community is, a countdown to the next tournament, featured games, the clip of the week and the latest news.
- **Games.** A library of every game your crew plays, with search, filters, sorting and favourites, loaded from a data file.
- **Game pages.** One page per game, showing its details, its upcoming tournaments and the players who main it.
- **Tournaments.** Upcoming, live and finished tournaments, with registration and a knockout bracket that runs from the first round to a champion.
- **Match night.** A live scoreboard for an evening of games that you run on one phone and share to your group chat at the end.
- **Leaderboard.** Worked out from a list of match results, sortable by any column, filterable by game.
- **Players and clans.** A profile for every player with their form and match history, and a page for every clan.

- **Join.** A sign-up form with proper validation.
- **News, About and a 404 page.**

It uses only HTML, CSS and JavaScript, and the data lives in JSON files inside your project.

What it does not have

Lobby has no server. When someone registers for a tournament on their phone, that registration is saved in their browser, on their phone only. Nobody else sees it. Real accounts, logins and a chat that everyone shares need a server and a database. That is the next thing to learn after this guide, and building Lobby first means you will know exactly why you need one.

Level 1

Structure

HTML · Days 1 to 14

HTML decides what every part of a page is: a heading, a list, a form, a table. You probably know the basic tags already. This level is about using the right ones for the right job, because every later level depends on it. CSS styles what your HTML says, and JavaScript finds things by what your HTML calls them.

There is no CSS in this level at all. Your pages will look plain, and that is on purpose. A page that makes sense with no styling is a page that is easy to style later.

By the end of this level

- a nine-page site with shared navigation
- a leaderboard table, a tournament list and a news page
- a sign-up form that checks its own input
- every page passing the W3C validator with no errors

Day 01 Start the Lobby

Phone

Every page needs the same few lines in its `head`. You will write them nine times in the next two weeks, so learn what each one does.

index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Lobby | Home of our gaming crew</title>
  <meta name="description" content="Tournaments, match nights and leaderboards for our gaming community.">
</head>
<body>
  <h1>Lobby</h1>
</body>
</html>
```

The `viewport` line stops phones from shrinking your page to fit a desktop-sized screen. Without it, everything looks tiny. The `description` is the short summary search engines often show under your link. The `lang` attribute tells screen readers which language to read the page in.

Your task

Make a folder called `lobby` with an `index.html` inside it. Type the head yourself instead of pasting it. Choose your community's name and use it in the title and the `h1`. Under the heading, write two short paragraphs: who the community is for, which games you play and where you usually meet (a WhatsApp group, a Discord server, somebody's parlour on Saturdays).

At the bottom of the body, add an HTML comment listing the pages you will build: home, games, tournaments, leaderboard and join.

Done when

- ☐ The page opens in your preview and shows your heading
- ☐ The browser tab shows your title
- ☐ Pasting your code into validator.w3.org (choose "Validate by direct input") gives no errors

Push further

Add a favicon. Make a small square picture of your logo, save it as `favicon.png` and link it with `<link rel="icon" href="favicon.png">`.

Day 02 The shape of the homepage

Phone

Pick elements for what they mean, not how they look. These are the ones that give a page its structure:

index.html

```
<header>
  <a href="index.html">Lobby</a>
  <nav>...</nav>
</header>
<main>
  <section>
    <h2>Games we play</h2>
    <article>...</article>
  </section>
  <aside>...</aside>
</main>
<footer>...</footer>
```

Use one **main** per page. A **section** is a part of the page with its own heading. An **article** is something that makes sense on its own, like a game card or a news post. An **aside** is related to the page but not the main point, like a box about the next tournament.

Your task

Rebuild your homepage with a real structure. The header holds your site name and a **nav** with five links (use `href="#"` for now). Inside **main**, add:

- a hero section with the **h1**, one line about your community and a "Join the crew" link
- a "Games we play" section with three articles, each with an **h3** and one sentence about why your crew plays that game
- an **aside** called "Next tournament" with the game, the date and the prize

The footer holds your social links and a copyright line.

Done when

- ☐ There is exactly one **h1**, sections use **h2** and cards use **h3**
- ☐ Every section has a heading
- ☐ Read with no styles, the page makes sense from top to bottom

Push further

Close the file and write the whole structure again from memory in a new file, in under fifteen minutes.

Day 03 Five pages, one navigation

Phone

A relative link points to a file based on where the current file is. `games.html` means the file next to this one. `pages/games.html` means inside a folder called pages. `../index.html` means one folder up.

index.html

```
<nav>
  <ul>
    <li><a href="index.html">Home</a></li>
    <li><a href="games.html" aria-current="page">Games</a></li>
    <li><a href="tournaments.html">Tournaments</a></li>
  </ul>
</nav>
```

`aria-current="page"` tells screen readers which link is the page they are on. In Level 2 you will also use it to highlight that link.

Your task

Create `games.html`, `tournaments.html`, `leaderboard.html` and `join.html`. Copy your header and footer into each one. Point every nav link at the right file, and on each page put `aria-current="page"` on the link for that page. Give each page its own title and description.

Done when

- ☐ From any page, you can reach every other page
- ☐ Every tab shows a different title
- ☐ Only one link per page has `aria-current`

If you are stuck

If a link shows an error page, check the spelling and the capital letters. `games.html` and `Games.html` are different files on most servers.

Push further

Move the four new pages into a `pages` folder and fix every link, including the ones pointing back home. Then decide whether it was worth it. Moving them back is allowed.

Day 04 Images that behave

Phone

An image needs four things: a source, alt text, a width and a height.

index.html

```
<figure>
  
  <figcaption>The final of our eFootball Friday league</figcaption>
</figure>
```

Width and height let the browser save space for the image before it loads, so the page does not jump around. `loading="lazy"` waits until the image is nearly on screen, which saves data. Alt text describes what is in the picture for anyone who cannot see it. "Image of game" tells them nothing.

Use screenshots you took yourself or placeholder images. Game cover art belongs to the publishers, so do not copy it from the internet for a site you will publish.

Your task

On `games.html`, build six game cards. Each card is an `article` with an image, an `h2` with the game's name and a sentence about the game. Keep all images in an `images` folder.

Done when

- ☐ Six cards, each with an image that loads
- ☐ Every image has alt text, a width and a height
- ☐ Every image after the first two has `loading="lazy"`

Push further

Resize your screenshots to 640 pixels wide before adding them. Most phone photo editors can do it, and squoosh.app does it in the browser. Note how much smaller the files get.

Day 05 Lists and details

Phone

There are three kinds of list. Use `ul` when the order does not matter, `ol` when it does, and `dl` for pairs of labels and values.

index.html

```
<dl>
  <dt>Genre</dt>
  <dd>Football</dd>
  <dt>Platforms</dt>
  <dd>Mobile, PlayStation</dd>
  <dt>Players per match</dt>
  <dd>2</dd>
</dl>
```

A list inside a list goes inside an `li`, not between two of them.

Your task

Under each game card, add a `dl` with the genre, the platforms, the number of players per match and where your crew plays it. On `tournaments.html`, add a rules section with an `ol` of at least eight rules. One rule must have a nested list, such as the rule about what happens when someone disconnects.

Done when

- ☐ Every card has a `dl` with at least four pairs
- ☐ The rules use `ol`, because their order matters
- ☐ The nested list sits inside an `li`

Push further

Add a "What to bring" list for in-person events: controller, charger, power bank, and anything else your crew always forgets.

Day 06 The leaderboard table

Phone

Use a table for data that has rows and columns, never for page layout.

index.html

```
<table>
  <caption>Season 1 standings</caption>
  <thead>
    <tr>
      <th scope="col">Rank</th>
      <th scope="col">Gamer tag</th>
      <th scope="col">Points</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>1</td>
      <th scope="row">Ada_Snipes</th>
      <td>58</td>
    </tr>
  </tbody>
</table>
```

`scope` tells screen readers whether a heading belongs to its column or its row, so they can read "Points: 58" instead of just "58".

Your task

On `leaderboard.html`, build a table of ten players with these columns: rank, gamer tag, main game, wins, losses and points. Make each gamer tag a row heading. Add a caption, and a `tfoot` with the date it was last updated in a `time` element: `<time datetime="2026-10-03">3 October</time>`.

Done when

- ☐ The caption describes the table
- ☐ Column headings use `scope="col"` and gamer tags use `scope="row"`
- ☐ The `time` element has a `datetime` in year-month-day format

Push further

Add a second table with two column groups, "This month" and "All time", each covering wins and points. Use `colspan` on the group headings.

Day 07 The join form

Phone

Every input needs a `label`. Match the label's `for` to the input's `id`, and tapping the label then puts the cursor in the input, which is a bigger target on a phone.

`index.html`

```
<label for="phone">Phone number</label>
<input id="phone" name="phone" type="tel">
```

The `type` changes the keyboard people see. `tel` gives a number pad, `email` adds the @ key and `number` gives digits. That alone makes a form easier to fill in on a phone.

Your task

On `join.html`, build a sign-up form with a gamer tag, full name, email, phone number, age, main game (a `select` with at least six games), platform (radio buttons for mobile, PC, PlayStation, Xbox and other), how they heard about you (a `select`), a short bio (`textarea`), an "I agree to the rules" checkbox and a submit button.

Done when

- ☐ Every input has a visible label that focuses it when tapped
- ☐ The phone field brings up a number pad on your phone
- ☐ All platform radio buttons share one `name`, so only one can be picked

Push further

Swap the main game `select` for a text input with a `datalist`, so people can pick from your list or type a game you did not think of.

Day 08 Validation with plain HTML

Phone

HTML can check a form before it is sent, without any JavaScript.

index.html

```
<label for="tag">Gamer tag</label>
<input id="tag" name="tag" required minlength="3" maxlength="16"
      pattern="[A-Za-z0-9_]+" title="Letters, numbers and underscores only">

<label for="phone">Phone number</label>
<input id="phone" name="phone" type="tel" required
      pattern="0[789][01][0-9]{8}" placeholder="08031234567">
```

That phone pattern accepts eleven-digit numbers starting with 070, 071, 080, 081, 090 or 091. **fieldset** and **legend** group related fields under a heading.

Your task

Add rules to every field that needs one: required fields, a gamer tag of 3 to 16 letters, numbers or underscores, a Nigerian mobile number, and an age from 13 to 99. The rules checkbox must be required. Group the form into three fieldsets: "About you", "Your gaming" and "The rules".

Done when

- ☐ Submitting an empty form shows the browser's error messages
- ☐ "ab" is rejected as a gamer tag, and so is "king tunde" with a space
- ☐ "12345" is rejected as a phone number, and so is an age of 12

Push further

Accept numbers written with +234 as well. The pattern `(0|\+234)[789][01][0-9]{8}` takes both.

Day 09 Make it work for everyone

Laptop recommended

About one person in six lives with some kind of disability. Many use a keyboard instead of a mouse or a screen reader instead of their eyes. Most of what they need comes free if your HTML is correct.

index.html

```
<body>
  <a href="#main" class="skip-link">Skip to content</a>
  <header>...</header>
  <main id="main">...</main>
</body>
```

A skip link lets keyboard users jump past the navigation on every page. Headings should go in order without skipping levels. Link text should make sense on its own: "See the tournament rules", not "click here".

Today's keyboard test needs a keyboard. Use a laptop, or connect a Bluetooth keyboard to your phone. Without either, do every other check today and come back to the Tab test later.

Your task

Go through all five pages. Add a skip link as the first thing in each **body**, and give each **main** the id it points to. Fix any heading that skips a level. Rewrite any link text that would not make sense read alone. Then press Tab through every page from top to bottom and write down anything you could not reach or could not see.

Done when

- ☐ The skip link is the first thing Tab reaches on every page
- ☐ Every link, button and input can be reached with Tab
- ☐ No heading level is skipped and no link says "click here"

Push further

Turn on your phone's screen reader, TalkBack on Android or VoiceOver on iPhone, and listen to your homepage for two minutes. Write down one thing that surprised you. Search for how to turn it off before you turn it on.

Day 10 Clips and embeds

Phone

To embed a YouTube video, use an **iframe** with a **title** that says what the video is. Screen readers announce the title, and without it they just say "frame".

index.html

```
<iframe src="https://www.youtube.com/embed/VIDEO_ID"
  title="Ada_Snipes' winning goal in the season final"
  width="560" height="315" loading="lazy" allowfullscreen></iframe>
<p><a href="https://www.youtube.com/watch?v=VIDEO_ID">Watch the clip on YouTube</a></p>
```

For your own video files, use **video** with **controls** so people can pause it, and a **poster** image that shows before it plays. Embeds need internet to load.

Your task

Add a "Clip of the week" section to your homepage with a YouTube embed of a gaming highlight from your community, a title describing it and a link to the video underneath. On **games.html**, add one short video file (under 5 MB) with controls and a poster image.

Done when

- ☐ The **iframe** has a title that describes the clip
- ☐ The video has controls and shows a poster before playing
- ☐ The link under the embed opens the same video

Push further

Record a ten-second clip of your own gameplay and use it instead of the YouTube embed.

The whole Lobby in HTML

Phone

The brief

Four days to turn five rough pages into a complete site you would be happy to show someone, still with no CSS. Plain HTML that reads well is the foundation everything else sits on, so do this week properly.

Must have

- ☐ `tournaments.html` lists three upcoming tournaments, each an `article` with the game, the date and time in a `time` element, the format (knockout or league), the entry fee, the prize in naira and a link to register
- ☐ `news.html` has three posts, each an `article` with a title, author, date in a `time` element and two paragraphs
- ☐ `about.html` says who runs the community, how to join the group chat, and has a code of conduct as a list
- ☐ `404.html` exists and links home
- ☐ every page has the same header, navigation and footer, with `aria-current` on the right link
- ☐ every page passes the W3C validator with no errors

Nice to have

- a "Hall of fame" table on `about.html` with past champions
- a contact form on `about.html` built with what you learned on days 7 and 8

Day by day

- | | |
|---------------|---|
| Day 11 | Tournaments page: three events with all their details. Use games your crew really plays and dates in the next few months. |
| <hr/> | |
| Day 12 | News page with three posts, and a single post page (<code>post-1.html</code>) that the first headline links to. |
| <hr/> | |
| Day 13 | About page and 404 page. Then go through every page and make the headers and footers match exactly. |
| <hr/> | |
| Day 14 | Validate every page, fix every error and click every link. Add a short changelog at the bottom of <code>about.html</code> listing what you built in days 1 to 14. |

Done when

- ☐ nine pages, all reachable from the navigation or from another page
- ☐ zero validator errors on every page
- ☐ no broken links and no broken images

If you are stuck

Validator errors often come in a chain. Fix the first one and run it again. Half of the rest may disappear with it.

Level 2

Style

CSS · Days 15 to 37

CSS decides how everything looks and where it sits. Gaming sites tend to be dark, bold and full of cards, and by the end of this level yours will be too. It will also work on a cheap phone and a wide laptop screen alike, and it will be online at a link you can send your crew.

The habit this level builds is consistency. Choose your colours, sizes and spacing once, give them names, and use those names everywhere. Sites that look professional are mostly sites that repeat themselves carefully.

By the end of this level

- a design system of colours, type and spacing held in CSS variables
- responsive layouts built with flexbox and grid
- a dark theme and a light theme
- your site live on GitHub Pages

Day 15 Link a stylesheet and set the basics

[Phone](#)

Put all your styles in one file and link it from every page with `<link rel="stylesheet" href="css/styles.css">` inside the `head`.

When two rules disagree, the more specific selector wins. An id beats a class, and a class beats an element name. When they are equally specific, the one written later wins. Some properties, like `color` and `font-family`, are inherited, so setting them on `body` changes the whole page.

styles.css

```
*, ::before, ::after { box-sizing: border-box; }

body {
  margin: 0;
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  line-height: 1.6;
}

img { max-width: 100%; height: auto; display: block; }
```

Your task

Create `css/styles.css` and link it from all nine pages. Type in the base rules above. Then try an experiment on one paragraph: give it a class and an id, style `p`, `.intro` and `#welcome` with three different colours, and write a comment in your CSS explaining which one won and why.

Done when

- ☐ The base styles show up on every page
- ☐ Your comment correctly explains which colour won
- ☐ No page still has styles inside its HTML

Push further

Look up the `:where()` selector on MDN and write a comment explaining why it is useful for base styles.

Day 16 A colour system

Phone

Store your colours in custom properties (CSS variables) on `:root`, then use them with `var()`. Change a colour in one place and it changes everywhere.

styles.css

```
:root {
  --bg: #0f1115;
  --surface: #181b22;
  --text: #e9ecf1;
  --muted: #9aa3b2;
  --accent: #39d98a;
  --danger: #ff5a5f;
  --live: #ff3d71;
}

body { background: var(--bg); color: var(--text); }
```

Text has to stand out from its background. The standard to aim for is a contrast ratio of at least 4.5 to 1 for normal text.

Your task

Choose seven colours for your brand: a background, a surface for cards, text, muted text, an accent, a danger colour, and a live colour used only for things happening right now, like a tournament in progress. Check your text against your background at webaim.org/resources/contrastchecker, which works in a phone browser. Apply the colours across the whole site using variables only.

Done when

- ☐ Hex colour codes appear only inside `:root`
- ☐ Body text on the background passes 4.5 to 1
- ☐ Links look different from normal text

Push further

Add `--accent-hover`, a slightly lighter version of your accent, and use it for links and buttons when they are hovered or focused.

Day 17 Type that reads well

[Phone](#)

Size text in `rem`, which is relative to the reader's own font setting. Use `clamp()` for headings so they grow with the screen but never become too small or too big. Keep paragraphs to around 65 characters wide, because long lines are tiring to read.

styles.css

```
h1 { font-size: clamp(2rem, 6vw, 3.5rem); line-height: 1.1; }
h2 { font-size: clamp(1.5rem, 4vw, 2.25rem); }
p { max-width: 65ch; }
```

Google Fonts (fonts.google.com) gives you free fonts with a `link` tag to paste into your `head`. They need internet every time the page loads unless the browser has saved them.

Your task

Pick one bold display font from Google Fonts for headings and keep the system font for body text. Set sizes for `h1`, `h2`, `h3`, body text and small text. Limit the width of paragraphs.

Done when

- ☐ Headings use your display font and body text uses the system font
- ☐ No font size anywhere is set in `px`
- ☐ No paragraph runs wider than about 65 characters

Push further

Download the font files, put them in a `fonts` folder and load them with `@font-face`, so your site no longer needs Google to show your headings.

Day 18 Spacing with the box model

[Phone](#)

Every element is a box: content, then `padding` around it, then a `border`, then `margin` outside. With `box-sizing: border-box` (from day 15), `width` includes the padding and border, which is much easier to reason about.

Pick a spacing scale and use nothing else:

styles.css

```
:root {
  --space-1: .25rem; --space-2: .5rem; --space-3: 1rem;
  --space-4: 1.5rem; --space-5: 2.5rem; --space-6: 4rem;
}

section { padding-block: var(--space-6); }
.card { padding: var(--space-4); }
```

Your task

Add a spacing scale and use it for every margin, padding and gap on the site. Give the game cards a surface background, padding and a thin border in your muted colour.

Done when

- ☐ Every margin and padding uses a `--space` variable
- ☐ Sections have the same breathing room on every page
- ☐ Nothing touches the edge of the screen on a phone

Push further

Open Eruda's Elements tab (or the laptop developer tools), inspect one card and find the diagram of its box model.

Day 19 The centred container

Phone

Most sites keep their content in a centred column so lines never get too long on wide screens. One line does it:

styles.css

```
.container {  
  width: min(100% - 2rem, 70rem);  
  margin-inline: auto;  
}
```

`min()` picks the smaller of the two, so on a phone the content is the screen width minus 1rem each side, and on a laptop it stops growing at 70rem. `margin-inline: auto` centres it.

Your task

Wrap the content of your header, each section and your footer in a `.container`. Make the footer links `display: inline-block` with some padding, so they are easier to tap.

Done when

- ☐ At any width, text never touches the edge of the screen
- ☐ On a wide screen, the content sits in the middle and stops at 70rem
- ☐ Footer links are easy to tap with a thumb

Push further

Add a narrower `.container-narrow` for news posts, around 42rem, so long articles are easier to read.

Day 20 Flexbox: the header

[Phone](#)

Flexbox lays things out in a row or a column and handles the spacing between them.

styles.css

```
.site-header .container {
  display: flex;
  align-items: center;
  justify-content: space-between;
  flex-wrap: wrap;
  gap: var(--space-3);
}

nav ul { display: flex; gap: var(--space-3); list-style: none; margin: 0; padding: 0; }

nav [aria-current="page"] { color: var(--accent); }
```

`justify-content` spaces things along the row and `align-items` lines them up across it. The last rule uses the `aria-current` you added on day 3 to style the current page's link.

Your task

Lay out the header with the site name on the left and the navigation on the right, wrapping underneath on narrow screens. Remove the underlines from nav links, give them padding, and make the current page's link stand out.

Done when

- ☐ On a wide screen, the header fits on one line
- ☐ On a phone, it wraps neatly without anything overlapping
- ☐ Each nav link is at least about 44 pixels tall, so it is easy to tap

Push further

Make a logo from your site name using only CSS: a bold font, a letter-spacing tweak and a small block of accent colour.

Day 21 Flexbox: cards that line up

[Phone](#)

Cards in a row look untidy when their buttons sit at different heights. Make each card a column and push the button to the bottom:

styles.css

```
.cards { display: flex; flex-wrap: wrap; gap: var(--space-4); }
.card { flex: 1 1 16rem; display: flex; flex-direction: column; }
.card .btn { margin-top: auto; }
```

`flex: 1 1 16rem` means grow and shrink as needed, starting from 16rem wide. `margin-top: auto` takes up all the spare space above the button.

Your task

Make the "Games we play" cards on your homepage a wrapping row. Inside each card, stack the image, the title, the text and a "See tournaments" link, with the link always at the bottom even when one card has more text than the others.

Done when

- ☐ Cards in the same row are the same height
- ☐ The links line up along the bottom of the row
- ☐ On a phone, each card takes the full width

Push further

Add a `.card--featured` variant that is twice as wide as the others on large screens.

Day 22 Grid: the game library

Phone

Grid lays things out in rows and columns at the same time. This one line makes a responsive gallery with no media queries at all:

styles.css

```
.game-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(14rem, 1fr));  
  gap: var(--space-4);  
}
```

It says: fit in as many columns as you can, each at least 14rem wide, and share out any spare space.

Your task

Turn the cards on `games.html` into a grid that shows one column on a phone, two or three on a tablet and four on a laptop, without writing a media query.

Done when

- ☐ Rotating your phone or resizing the window changes the number of columns
- ☐ Nothing scrolls sideways at any width
- ☐ Cards keep the styles from days 18 and 21

Push further

Add six more games so you have twelve, and check the grid still looks right.

Day 23 Grid: the homepage layout

Phone

`grid-template-areas` lets you draw the layout with words. Change the drawing in a media query and the whole page rearranges.

```
styles.css

.home {
  display: grid;
  gap: var(--space-5);
  grid-template-areas: "hero" "games" "next" "news";
}

@media (min-width: 60rem) {
  .home {
    grid-template-columns: 2fr 1fr;
    grid-template-areas:
      "hero  hero"
      "games next"
      "news  next";
  }
}

.hero { grid-area: hero; }
.next-tournament { grid-area: next; }
```

Your task

Lay out your homepage with grid areas. On a phone everything stacks in a sensible order. On a wide screen, the next tournament box becomes a sidebar next to the games and the news.

Done when

- ☐ The phone order makes sense read from top to bottom
- ☐ On a wide screen, the sidebar sits on the right
- ☐ Nothing overlaps at any width

If you are stuck

To see the wide layout on a phone, turn it sideways or switch your browser to "Desktop site".

Push further

Make the sidebar stay on screen while you scroll, with `position: sticky; top: var(--space-4);`.

Day 24 A leaderboard people want to read

Phone

Tables need a few extra rules to read well. Numbers should line up on the right, and `tabular-nums` makes every digit the same width so columns do not wobble.

styles.css

```
table { border-collapse: collapse; width: 100%; }
td, th { padding: var(--space-2) var(--space-3); text-align: left; }
td.num { text-align: right; font-variant-numeric: tabular-nums; }
tbody tr:nth-child(even) { background: var(--surface); }
tbody tr:nth-child(1) { background: rgb(255 200 0 / .15); }

.table-wrap { overflow-x: auto; }
```

A wide table on a phone should scroll sideways inside its own box, not make the whole page scroll.

Your task

Style your leaderboard: striped rows, numbers aligned right with tabular figures, and the top three rows tinted gold, silver and bronze. Wrap the table in a `.table-wrap` so it scrolls on its own on a phone.

Done when

- ☐ The number columns line up
- ☐ The top three rows stand out
- ☐ On a phone, the page itself never scrolls sideways

Push further

Keep the heading row visible while the table scrolls, using `position: sticky` on the `th` elements.

Browsers give form fields a plain look you can replace.

styles.css

```
input, select, textarea {
  font: inherit;
  font-size: 1rem;
  width: 100%;
  padding: .7rem .8rem;
  color: var(--text);
  background: var(--surface);
  border: 1px solid var(--muted);
  border-radius: .4rem;
}

input:focus-visible { outline: 3px solid var(--accent); outline-offset: 2px; }
input:user-invalid { border-color: var(--danger); }
input[type="checkbox"], input[type="radio"] { accent-color: var(--accent); width: auto; }
```

`:user-invalid` only turns red after the person has typed something wrong, not the moment the page loads. Keep inputs at 16 pixels or bigger, because iPhones zoom into any field with smaller text.

Your task

Style the join form. Labels go above their inputs, inputs fill the width on a phone, and short fields sit side by side on wide screens. Focus must always be visible, errors should only show after someone has typed, and the submit button should be the most obvious thing on the page.

Done when

- ☐ You can always see which field has focus
- ☐ A field only turns red after you type something wrong in it
- ☐ Tapping a field on a phone does not zoom the page

Push further

Style each fieldset as a clear group with its legend as a small heading.

Day 26 Positioning: sticky, absolute and fixed

Phone

`position: sticky` keeps something on screen as you scroll past it. `position: absolute` places something relative to the nearest parent that has a position of its own. `position: fixed` pins something to the screen.

styles.css

```
.site-header { position: sticky; top: 0; z-index: 10; background: var(--bg); }

.tournament-card { position: relative; }
.badge-live {
  position: absolute;
  top: var(--space-2);
  right: var(--space-2);
}
```

If an absolutely positioned badge flies off to the corner of the page, its parent is missing `position: relative`.

Your task

Make your header sticky. Add a "LIVE" badge in the top corner of any tournament card with the class `is-live`. Add a fixed "Back to top" link in the bottom corner of long pages, pointing at an element with `id="top"`.

Done when

- ☐ The header stays visible while you scroll
- ☐ Each badge sits in the corner of its own card
- ☐ The back to top link does not cover any text on a phone

Push further

Show the back to top link only on narrow screens, where scrolling back up takes longest.

Day 27 Hover, focus and small details

Phone

`::before` and `::after` add decoration without adding HTML. They need a `content` property, even an empty one.

styles.css

```
nav a { position: relative; }
nav a::after {
  content: "";
  position: absolute;
  left: 0; bottom: -.2rem;
  width: 0; height: 2px;
  background: var(--accent);
  transition: width .2s;
}
nav a:hover::after,
nav a:focus-visible::after { width: 100%; }
```

Phones have no hover, and keyboard users do not hover either. Anything that appears on hover should also appear on `:focus-visible`, and nothing important should only appear on hover.

Your task

Add the animated underline to your nav links. On the leaderboard, put a small coloured circle with the rank number before each of the top three gamer tags using `::before`.

Done when

- ☐ The underline appears on hover and on keyboard focus
- ☐ The rank circles come from CSS, not from extra HTML
- ☐ The page still makes sense with every decoration removed

Push further

Style `::selection` so highlighted text uses your accent colour.

Day 28 Mobile first, then bigger screens

Laptop recommended

Write your base styles for phones, then add `min-width` media queries for bigger screens. Phones then download simple styles, and each breakpoint only adds what the bigger screen needs.

styles.css

```
.stats { display: grid; gap: var(--space-3); }

@media (min-width: 40rem) {
  .stats { grid-template-columns: repeat(2, 1fr); }
}

@media (min-width: 64rem) {
  .stats { grid-template-columns: repeat(4, 1fr); }
}
```

Use the same two or three breakpoints on every page, so the whole site changes shape at the same widths.

A phone can only show you phone widths. The laptop's device toolbar (Ctrl+Shift+M in the developer tools) shows every width in seconds. On a phone, turning sideways and "Desktop site" mode give you a rough idea.

Your task

Go through every page. Rewrite any `max-width` media queries as `min-width` ones at 40rem and 64rem. Then check every page at 320, 768 and 1440 pixels wide and fix whatever breaks.

Done when

- ☐ Nothing scrolls sideways at 320 pixels wide
- ☐ Nothing looks stretched at 1440 pixels
- ☐ Every page changes layout at the same breakpoints

Push further

Add a print stylesheet for the tournament rules, so people can print them without the navigation and dark background.

Day 29 Images in CSS

[Phone](#)

object-fit: **cover** crops an image to fill its box without squashing it. **aspect-ratio** gives every image the same shape. For text on top of a photo, darken the photo with a gradient so the text stays readable.

```
styles.css

.card img { aspect-ratio: 16 / 9; object-fit: cover; width: 100%; }

.hero {
  background:
    linear-gradient(to right, rgb(0 0 0 / .85), rgb(0 0 0 / .2)),
    url("../images/hero.jpg") center / cover;
}
```

Notice the `..` in the image path. The path is relative to the CSS file, not the HTML page.

Your task

Give your homepage hero a background image with a dark gradient over it. Make every game card's image the same 16 by 9 shape, however big or small the original is.

Done when

- ☐ The hero text is easy to read over the busiest part of the image
- ☐ Every card image is the same shape and nothing is squashed
- ☐ The hero image loads (check the `..` if it does not)

Push further

Use a media query to load a smaller hero image on phones, to save your visitors' data.

Day 30 Motion with transitions

[Phone](#)

A **transition** animates the change from one style to another. Keep it short, under about 300 milliseconds, so the site feels quick. Some people get dizzy or sick from movement on screen and ask their phone to reduce it, so respect that setting.

```
styles.css

.card { transition: transform .2s ease, box-shadow .2s ease; }
.card:hover, .card:focus-within { transform: translateY(-4px); }
.btn:active { transform: scale(.97); }

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { transition: none !important; animation: none !important; }
}
```

Your task

Make cards lift slightly on hover and when something inside them has focus. Make buttons shrink a little when pressed. Fade link colours instead of snapping them. Wrap all of it in a reduced-motion rule.

Done when

- ☐ Every animation lasts less than 300 milliseconds
- ☐ Turning on reduced motion stops all of it (Android: Remove animations; iPhone: Reduce Motion, both in Accessibility settings)
- ☐ Nothing moves on its own when the page loads

Push further

Give the main button a soft glow in your accent colour using `box-shadow` when it is hovered or focused.

Day 31 Keyframe animations

Phone

`@keyframes` describes an animation step by step, and `animation` plays it.

styles.css

```
@keyframes pulse {
  0%, 100% { opacity: 1; }
  50%      { opacity: .35; }
}

.live-dot {
  width: .6rem; height: .6rem; border-radius: 50%;
  background: var(--live);
  animation: pulse 1.4s ease-in-out infinite;
}
```

Your task

Add a pulsing dot inside the LIVE badge from day 26. Then build a skeleton card: a grey placeholder shaped like a game card, with a light band that sweeps across it while content loads. Keep it in a page called `skeleton.html` for now. You will use it in Level 3 while data loads.

Done when

- ☐ The dot pulses smoothly and slowly
- ☐ The skeleton is the same shape as a real card
- ☐ Both stop when reduced motion is on

Push further

Build a spinning loading ring from a single element with a coloured top border.

Lobby gets its look

Phone

The brief

Six days to make every page consistent, responsive and themed, and to put the site online by the end of the week. Send your crew the link on day 37.

Must have

- ☐ one design system of colour, type and spacing variables, used everywhere
- ☐ reusable components with consistent class names, at least `.btn`, `.card`, `.badge`, `.container` and `.section-title`
- ☐ every page working from 320 pixels wide to 1440 pixels wide
- ☐ a dark theme by default and a light theme using `prefers-color-scheme`
- ☐ the site live on GitHub Pages at a link you can send to people

Nice to have

- a style guide page, `styleguide.html`, showing every component
- a styled 404 page that matches the rest of the site

Day by day

- | | |
|--------|---|
| Day 32 | Find repeated styles and turn them into components. Write a comment at the top of <code>styles.css</code> listing every component and what it is for. |
| Day 33 | Light theme: redefine your colour variables inside <code>@media (prefers-color-scheme: light)</code> . Check the contrast again in both themes. |
| Day 34 | Style the tournaments, news, about and 404 pages using only your existing components. If you need a new one, add it to the list first. |
| Day 35 | Responsive check of every page at 320, 375, 768, 1024 and 1440 pixels wide. |
| Day 36 | Build the style guide page, or finish anything left on the must-have list. |
| Day 37 | Put the site online (below) and send the link to three people in your group. Get one piece of feedback from each of them. |

New ideas this week

Putting your site online from a phone

1. Make a free account at github.com in your phone browser.
2. Create a new repository called `Lobby` and set it to Public.
3. Upload your files with "Add file" then "Upload files". If you cannot see the button, switch your browser to "Desktop site".
4. Phone browsers usually cannot upload a whole folder at once. Upload the top-level files first. For each folder, create it by adding a new file with the folder in its name, such as `css/styles.css`, then open the folder and upload the rest of its files into it.
5. Go to Settings, then Pages. Under Branch, choose `main` and `/ (root)`, then Save.
6. After a minute or two, your site is at <https://your-username.github.io/lobby/>.

On a laptop, use Git instead (see Setting up on a laptop). It keeps the folders for you and is much quicker.

Done when

- ☐ the live link works on mobile data, not just on your wifi
- ☐ every page looks finished in both themes
- ☐ three people have seen it and you have written down what they said

If you are stuck

If GitHub Pages shows a 404, check that your homepage is named `index.html` in lowercase and sits at the top of the repository, not inside a folder.

Level 3

Behaviour

JavaScript · Days 38 to 72

Your site looks finished, but nothing on it responds to anything. In this level it starts to: the leaderboard builds itself from data, forms check what people type, favourites are remembered, a countdown ticks towards the next tournament, and match night gets a live scoreboard you can share to your group chat.

From day 64, your data moves into JSON files that your pages load with `fetch`. That is also where the "Phone + server" label starts to matter, so reread its explanation on the phone setup page before you get there.

By the end of this level

- pages that build themselves from data
- forms with live checks, filters, dialogs and a mobile menu
- a match night scoreboard that saves itself and shares its results
- data loaded from JSON files and from a real API

Day 38 JavaScript on every page

Phone

Load one script on every page, from the `head`, with `defer`:

```
index.html

<script src="js/site.js" defer></script>
```

`defer` means download the file straight away but only run it once the whole page has been read. Without it, your code can run before the elements it is looking for exist.

You need a console to see errors and messages. On a phone, add Eruda (see Setting up on a phone) above this line while you build.

Your task

Create `js/site.js` and load it on all nine pages. Make it do two things: log a welcome message that includes the page's title (`document.title`), and fill in the year in your footer automatically, so you never have to update it. Put an empty `<span class="year"></span>` in each footer, then:

script.js

```
document.querySelector(".year").textContent = new Date().getFullYear();
```

Done when

- ☐ The console shows the welcome message on every page, with the right title
- ☐ The footer shows this year without you typing it
- ☐ No errors appear in the console on any page

Push further

Log how many links each page has, using `document.querySelectorAll("a").length`.

Day 39 Numbers, strings and match stats

[Phone](#)

You will do a lot of small sums with game stats. `toFixed()` rounds a number to a set number of decimal places, but it gives you back a string. Template literals (backticks) drop values into text.

script.js

```
const wins = 18;
const draws = 4;
const losses = 6;
const kills = 212;
const deaths = 97;

const played = wins + draws + losses;
const points = wins * 3 + draws;
const winRate = (wins / played * 100).toFixed(1);
const kd = (kills / deaths).toFixed(2);

console.log(`KING_TUNDE | ${wins}W ${draws}D ${losses}L | ${points} pts | ${winRate}%`);
```

That prints `KING_TUNDE | 18W 4D 6L | 58 pts | 64.3%`. Useful string methods: `toUpperCase()`, `trim()` (removes spaces at the ends) and `padStart()`.

Your task

Create `js/stats.js` and load it on the leaderboard page. Use variables for one player's tag (written in lowercase), wins, draws, losses, kills and deaths. Log: the total matches, the points, the win rate as a percentage with one decimal place, the K/D ratio with two, and a summary line like the one above with the tag in capitals. Then log their rank padded to two digits, so 7 becomes `07`.

Done when

- ☐ Every number matches what you get by working it out on paper
- ☐ The tag appears in capitals without you changing the variable
- ☐ Nothing prints `NaN`

If you are stuck

`NaN` means "not a number". One of your values is text or `undefined`. Log each variable with `typeof` in front of it to find which one.

Day 40 Decisions: ranking tiers

[Phone](#)

`if`, `else if` and `else` pick one path. When you check ranges, start from the top and work down, because the first true condition wins.

script.js

```
const points = 58;
let tier;

if (points >= 200) {
  tier = "Legend";
} else if (points >= 100) {
  tier = "Diamond";
} else if (points >= 50) {
  tier = "Gold";
} else if (points >= 20) {
  tier = "Silver";
} else {
  tier = "Bronze";
}
```

`&&` means and, `||` means or and `!` means not. Always compare with `===`, which checks the type as well as the value.

Your task

Write the tier code for a `points` variable. Then add two things: a message for negative points ("Points can't be negative"), and a line saying how many points the player needs to reach the next tier, like "42 points to Diamond". Legends get "Top tier". Test it by changing `points` to 0, 19, 20, 99, 100, 199, 200, 350 and -5.

Done when

- ☐ 19 gives Bronze, 20 gives Silver, 199 gives Diamond and 200 gives Legend
- ☐ The points to the next tier are right for every test value
- ☐ -5 shows the error message and nothing else

Day 41 Functions: your toolbox

[Phone](#)

A function wraps code up so you can reuse it with different inputs. `return` hands the answer back. A function that logs instead of returning is only useful for looking at.

script.js

```
function formatNaira(amount) {  
  return new Intl.NumberFormat("en-NG", {  
    style: "currency",  
    currency: "NGN",  
    maximumFractionDigits: 0  
  }).format(amount);  
}  
  
formatNaira(50000); // "₦50,000"
```

Arrow functions are a shorter way to write small ones: `const double = n => n * 2;`

Your task

Create `js/utils.js` and load it before your other scripts. Write these five functions and test each one with `console.log`:

- `formatNaira(amount)` from above
- `tierFor(points)`, using yesterday's code
- `winRate(wins, played)`, which returns 0 when `played` is 0
- `kdRatio(kills, deaths)`, which returns `kills` when `deaths` is 0
- `initials(tag)`, where `initials("King_Tunde")` gives "KT"

Then use `formatNaira` to show the prize on each tournament card.

Done when

- ☐ Every function returns a value and none of them log
- ☐ No test gives you `Infinity` or `NaN`, even with zeros
- ☐ The tournament prizes show with the naira sign and commas

If you are stuck

For `initials`, split the tag on the underscore with `tag.split("_")`, then take the first letter of each piece.

Push further

Add `plural(count, word)`, so `plural(1, "match")` gives "1 match" and `plural(3, "match")` gives "3 matches".

Day 42 Arrays and loops

[Phone](#)

An array holds a list. `for...of` runs some code for each item.

```
script.js

const tags = ["Ada_Snipes", "King_Tunde", "Zee_Ruthless", "BigBola", "Nnamdi99"];

for (const tag of tags) {
  console.log(tag);
}

tags.push("LagosLag"); // add to the end
tags.length;           // how many
tags.includes("BigBola"); // true or false
```

When you need the position too, use a counting loop: `for (let i = 0; i < tags.length; i++)`.

Your task

Make an array of twelve gamer tags. Write code that:

1. logs a numbered list, "1. Ada_Snipes" and so on
2. finds and logs the longest tag
3. counts how many tags contain a number, using `/[0-9]/.test(tag)`
4. builds a new array of every tag in capitals
5. checks whether a new tag is already taken, ignoring capitals, so "bigbola" counts as taken

Done when

- ☐ All five results are correct when you check them by eye
- ☐ The taken check says yes for "BIGBOLA" and no for "BigBolaX"
- ☐ The original array is unchanged at the end

Day 43 Objects and lists of objects

Phone

An object groups the details of one thing under names. A list of objects is the shape of nearly all real data.

script.js

```
const players = [  
  { tag: "Ada_Snipes", game: "eFootball", platform: "mobile",  
    wins: 21, draws: 3, losses: 4, kills: 0, deaths: 0, clan: "Lagos Lions" },  
  { tag: "King_Tunde", game: "Call of Duty: Mobile", platform: "mobile",  
    wins: 18, draws: 4, losses: 6, kills: 212, deaths: 97, clan: "Lagos Lions" }  
];  
  
console.log(players[1].tag); // "King_Tunde"
```

Your task

Create `js/data.js` with ten players, each with a tag, name, main game, platform, wins, draws, losses, kills, deaths and clan. Make the numbers realistic and different from each other. Loop through them and log a summary for each, using your functions from day 41. Then find the player with the most wins using a loop, without sorting.

Done when

- ☐ Ten players, every one with all ten details
- ☐ A summary line for each, with tier, win rate and K/D
- ☐ The most wins found by a loop, and correct

Arrays come with methods that replace most loops.

script.js

```
const points = p => p.wins * 3 + p.draws;

const mobile = players.filter(p => p.platform === "mobile");
const tags   = players.map(p => p.tag);
const ada    = players.find(p => p.tag === "Ada_Snipes");
const total  = players.reduce((sum, p) => sum + p.wins, 0);
const board  = [...players].sort((a, b) => points(b) - points(a));
```

`sort` changes the array it is called on, so copy it first with `[...players]`. `reduce` walks the list carrying a running result, which starts at the value after the comma.

Your task

Using only these methods, with no `for` loops today, work out:

1. the total number of matches played by everyone
2. the leaderboard order by points, with win rate breaking ties
3. every player in Diamond tier or above
4. the average K/D of players who have any kills
5. an object counting players per game, like `{ "eFootball": 4, "Call of Duty: Mobile": 3 }`

Done when

- ☐ Results match what yesterday's loops gave you
- ☐ The original `players` array is still in its original order
- ☐ No `for` loops anywhere in today's code

If you are stuck

For the tie-break, `points(b) - points(a) || winRate(...)` works because a difference of 0 counts as false, so JavaScript moves on to the part after `||`. For the count per game, start `reduce` with an empty object `{}` and add 1 to `counts[p.game]` each time.

Day 45 Build the leaderboard from data

Phone

To change the page from JavaScript, find an element, then change it or add to it.

script.js

```
const tbody = document.querySelector("#board tbody");

board.forEach((player, i) => {
  const row = document.createElement("tr");
  row.innerHTML = `<td>${i + 1}</td><th scope="row"></th><td class="num">${points(player)}</td>`;
  row.querySelector("th").textContent = player.tag;
  tbody.append(row);
});
```

`innerHTML` reads text as HTML, so anything a user typed could run as code on your page. Use `textContent` for anything that came from a person, like a gamer tag. Numbers you calculated yourself are safe in `innerHTML`.

Your task

Delete the hand-typed rows from your leaderboard and build them from the players array instead, sorted by yesterday's order. Show rank, tag, game, wins, draws, losses, points and tier. Keep the gold, silver and bronze styles from day 24 working.

Done when

- ☐ Changing someone's wins in `data.js` changes the table after a refresh
- ☐ Gamer tags go in with `textContent`
- ☐ The table still has its caption and its `scope` attributes

Push further

Build all the rows inside a `DocumentFragment` and add them to the table in one go.

Day 46 Game cards from a template

Phone

A `template` holds HTML the browser does not show. Copy it once per item, fill in the copy and add it to the page.

index.html

```
<template id="game-card">
  <article class="card">
    <img width="640" height="360" loading="lazy">
    <h2></h2>
    <p class="genre"></p>
    <p class="desc"></p>
  </article>
</template>
```

script.js

```
const template = document.querySelector("#game-card");

function renderGame(game) {
  const card = template.content.cloneNode(true);
  card.querySelector("img").src = game.image;
  card.querySelector("img").alt = game.imageAlt;
  card.querySelector("h2").textContent = game.title;
  card.querySelector(".genre").textContent = game.genre;
  card.querySelector(".desc").textContent = game.description;
  return card;
}
```

Your task

Add a `games` array to `data.js` with at least twelve games. Give each an id, title, genre, platforms (an array), max players, image, image alt text and description. Replace the hand-written cards on `games.html` with cards built from the template.

Done when

- ☐ Twelve cards, all built from data
- ☐ Adding a game to the array adds a card after a refresh
- ☐ The grid from day 22 still works

Day 47 Clicks and a mobile menu

Phone

`addEventListener` runs a function when something happens. A button that shows and hides something should say whether it is open, using `aria-expanded`.

index.html

```
<button class="menu-toggle" aria-expanded="false" aria-controls="site-menu">Menu</button>
<ul id="site-menu">...</ul>
```

script.js

```
const toggle = document.querySelector(".menu-toggle");
const menu = document.querySelector("#site-menu");

toggle.addEventListener("click", () => {
  const open = toggle.getAttribute("aria-expanded") === "true";
  toggle.setAttribute("aria-expanded", String(!open));
  menu.classList.toggle("is-open", !open);
});
```

Your task

On narrow screens, hide your navigation behind a Menu button that opens and closes it. Use CSS to hide the list unless it has the `is-open` class, and to always show it on wide screens. On `games.html`, add a "Compact view" button that switches the grid to a tight list and back.

Done when

- ☐ The menu opens and closes with a tap and with the keyboard
- ☐ `aria-expanded` changes between true and false as it opens and closes
- ☐ The compact view shows the same games, just laid out differently

Push further

Close the menu when someone taps a link inside it or presses Escape.

Day 48 One listener for many buttons

[Phone](#)

Instead of adding a listener to every card, add one to the grid and check what was clicked. This is called event delegation, and it also works for cards added later.

```
script.js

grid.addEventListener("click", (event) => {
  const button = event.target.closest(".save");
  if (!button) return;

  const saved = button.getAttribute("aria-pressed") === "true";
  button.setAttribute("aria-pressed", String(!saved));
  button.textContent = saved ? "Save" : "Saved";
});
```

`data-` attributes store extra information on an element. `<button data-id="efootball">` is read in JavaScript as `button.dataset.id`.

Your task

Add a Save button to every game card, with the game's id in a `data-id` attribute and `aria-pressed="false"`. One listener on the grid handles every button. Show the number of saved games in the page heading, like "Games (3 saved)".

Done when

- ☐ There is only one click listener, on the grid
- ☐ Each button switches between Save and Saved
- ☐ The count in the heading is always right

Push further

Style saved cards differently using the selector `.card:has(.save[aria-pressed="true"])`.

Day 49 Handling the join form

[Phone](#)

When a form is submitted, the browser normally reloads the page. `preventDefault()` stops that, so you can handle the data yourself. `FormData` reads every field at once, using the `name` attributes.

```
script.js

form.addEventListener("submit", (event) => {
  event.preventDefault();
  const data = Object.fromEntries(new FormData(form));
  console.log(data.tag, data.game, data.platform);
});
```

The HTML checks from day 8 still run first. The submit event only fires once every field is valid.

Your task

When the join form is submitted, hide it and show a welcome card instead: "Welcome to Lobby, KING_TUNDE", with their main game, platform and a starting tier of Bronze. Add an "Edit my details" button that hides the card and brings the form back with everything still filled in.

Done when

- ☐ The page does not reload when you submit
- ☐ Everything the person typed goes into the page with `textContent`
- ☐ Editing brings the form back with their answers in it

If you are stuck

Nothing is sent anywhere yet, because there is no server. That is expected. It is covered after this guide.

Day 50 Live checks while people type

[Phone](#)

The `input` event fires on every key press. `setCustomValidity()` sets your own error message. An empty message means the field is valid again.

script.js

```
const taken = players.map(p => p.tag.toLowerCase());

tagInput.addEventListener("input", () => {
  const isTaken = taken.includes(tagInput.value.trim().toLowerCase());
  tagInput.setCustomValidity(isTaken ? "That gamer tag is already taken" : "");
  tagStatus.textContent = isTaken ? "Taken" : "Available";
});
```

Your task

As someone types a gamer tag, show "Available" or "Taken" underneath, checked against your players and ignoring capitals. A taken tag must stop the form from submitting. Add a character counter under the bio, "87 / 160", that turns your danger colour after 140 characters, and set the bio's `maxLength` to 160.

Done when

- ☐ A taken tag cannot be submitted, whatever capitals are used
- ☐ The message clears as soon as the tag is changed to a free one
- ☐ The counter updates with every key press

Push further

Also block tags that start with a number or contain two underscores in a row.

Day 51 Filter buttons

Phone

Filter buttons are buttons that show only some of the cards. `aria-pressed` tells screen readers which one is active, and the `hidden` attribute hides an element.

script.js

```
filters.addEventListener("click", (event) => {
  const button = event.target.closest("button[data-filter]");
  if (!button) return;

  for (const b of filters.querySelectorAll("button")) {
    b.setAttribute("aria-pressed", String(b === button));
  }

  for (const card of cards) {
    const show = button.dataset.filter === "all" || card.dataset.status ===
button.dataset.filter;
    card.hidden = !show;
  }
});
```

Your task

Give each tournament card a `data-status` of upcoming, live or finished. Add filter buttons for All, Upcoming, Live and Finished. When a filter has no tournaments, show a message such as "No live tournaments right now".

Done when

- ☐ Exactly one button is pressed at a time
- ☐ The empty message appears when a filter matches nothing
- ☐ Everything works with the keyboard

If you are stuck

If cards will not hide, your CSS is probably giving them `display: flex` or `display: grid`, which beats the `hidden` attribute. Add `[hidden] { display: none !important; }` to your stylesheet.

Day 52 A dialog for game details

[Phone](#)

The `dialog` element makes a pop-up that handles most of the hard parts for you. `showModal()` opens it, blocks the rest of the page and lets Escape close it.

index.html

```
<dialog id="game-dialog" aria-labelledby="dialog-title">
  <h2 id="dialog-title"></h2>
  <p class="dialog-desc"></p>
  <form method="dialog"><button>Close</button></form>
</dialog>
```

script.js

```
dialog.querySelector("h2").textContent = game.title;
dialog.showModal();
```

A button inside `form method="dialog"` closes the dialog without any JavaScript.

Your task

Add a Details button to each game card. Tapping it fills one shared dialog with that game's title, description, platforms and max players, then opens it. When the dialog closes, put focus back on the button that opened it.

Done when

- ☐ One dialog is reused for every game
- ☐ The Close button and Escape both close it
- ☐ Focus returns to the right Details button

If you are stuck

Save the button that was clicked in a variable before opening the dialog. Then listen for the dialog's `close` event and call `.focus()` on that button.

Day 53 Remembering things

[Phone](#)

`localStorage` keeps small pieces of text in the browser, even after it is closed. It only stores strings, so turn arrays and objects into text with `JSON.stringify` and back with `JSON.parse`.

script.js

```
function loadSaved() {
  return JSON.parse(localStorage.getItem("saved-games") || "[]");
}

function storeSaved(ids) {
  localStorage.setItem("saved-games", JSON.stringify(ids));
}
```

It is saved on that phone, in that browser only. Another phone will not see it.

Your task

Make saved games stay saved after a refresh. Then add a theme button that switches between dark and light and remembers the choice. Set `data-theme="light"` or `data-theme="dark"` on the `html` element, and in your CSS let `:root[data-theme="light"]` and `:root[data-theme="dark"]` override the system theme from boss week 2.

Done when

- ☐ Saved games survive closing and reopening the browser
- ☐ The theme choice survives too
- ☐ Clearing the site's data resets both

Push further

Add a "Clear my data" button on the about page.

Day 54 Dates and a live countdown

[Phone](#)

Write dates in ISO format with a time zone, so there is no doubt about when you mean. Lagos is one hour ahead of UTC, which is `+01:00`.

script.js

```
const start = new Date("2026-11-14T16:00:00+01:00");

function tick() {
  const ms = start - Date.now();
  if (ms <= 0) {
    out.textContent = "Live now";
    clearInterval(timer);
    return;
  }
  const s = Math.floor(ms / 1000);
  const d = Math.floor(s / 86400);
  const h = Math.floor(s % 86400 / 3600);
  const m = Math.floor(s % 3600 / 60);
  out.textContent = `Starts in ${d}d ${h}h ${m}m ${String(s % 60).padStart(2, "0")}s`;
}

const timer = setInterval(tick, 1000);
tick();
```

Your task

Add a countdown to your homepage's next tournament box, set to a date a few days from today. It updates every second, switches to "Live now" when the time comes, and shows the start time in Lagos time underneath using `start.toLocaleString("en-NG", { timeZone: "Africa/Lagos", dateStyle: "medium", timeStyle: "short" })`.

Done when

- ☐ The countdown is right to the second
- ☐ It changes to Live now at zero and stops ticking
- ☐ The start time shows in Lagos time on a phone set to any time zone

Push further

Set the date two minutes from now to watch it change over.

Day 55 A reaction-time mini game

Phone

`setTimeout` runs code once after a delay. `performance.now()` gives a precise time in milliseconds, so the difference between two readings is how long something took.

A game like this has states. Keep one variable saying which state you are in, and decide what a tap means based on it.

script.js

```
let state = "idle"; // "idle", "waiting" or "ready"

const delay = 1500 + Math.random() * 2500; // between 1.5 and 4 seconds
```

Your task

Build `arcade.html`. A big box says "Tap to start". Once tapped, it says "Wait for green", then after a random delay turns green and says "Tap!". Show how many milliseconds the player took. Tapping before it turns green says "Too soon!" and cancels the round. Save the player's best time in `localStorage` and show "Your best: 238 ms".

Done when

- ☐ The delay is different every round
- ☐ An early tap is caught and the green never appears for that round
- ☐ The best time survives a refresh and only goes down, never up

If you are stuck

To cancel the round on an early tap, save what `setTimeout` returns in a variable and pass it to `clearTimeout`.

Push further

Use the `pointerdown` event instead of `click` for a faster response, and show the average of five attempts with the slowest one dropped.

Day 56 Debugging like you mean it

Laptop recommended

When code does not work, read the error first. It has a type, a message and a file and line number.

`console.table(array)` shows a list of objects as a table, which is often faster than reading logs.

On a laptop, the Sources tab lets you set a breakpoint: click a line number, reload, and the code pauses there so you can look at every variable. The `debugger;` statement does the same from inside your code.

Breakpoints need desktop developer tools. On a phone you can still do today's task with `console.log` and Eruda, just more slowly.

Your task

This code has five bugs. Copy it into a new file, run it, and fix them one at a time. Next to each fix, write a comment saying what was wrong.

script.js

```
const players = [
  { tag: "Ada_Snipes", wins: 12, played: 20 },
  { tag: "King_Tunde", wins: "9", played: 15 },
  { tag: "Zee_Ruthless", wins: 0, played: 0 }
];

function winRate(p) {
  return (p.wins / p.played * 100).toFixed(1);
}

const totalWins = players.reduce((sum, p) => sum + p.wins, 0);
console.log("Total wins:", totalWins);

const best = players.sort((a, b) => a.wins - b.wins)[0];
console.log("Best player: " + best.Tag);

for (let i = 0; i <= players.length; i++) {
  console.log(players[i].tag, winRate(players[i]) + "%");
}
```

Done when

- ☐ It prints a total of 21 wins
- ☐ It names Ada_Snipes as the best player
- ☐ Every win rate is a real number, with no NaN, and there are no errors
- ☐ Five comments, one for each bug

If you are stuck

Fix them in the order they appear in the output: the total first, then the best player, then the loop.

Match night: a live scoreboard

Phone

The brief

Match night is when your crew gets together, in person or online, and plays a mini league in one evening. You are building the page that runs it: add tonight's players, record results as they happen, and keep a live table everyone can see on one phone. At the end, share the final table straight to your group chat.

Must have

- ☐ `matchnight.html` with a form to add tonight's players: between 2 and 16, with no duplicates
- ☐ a form to record a result with two players and two scores, where a player cannot play themselves
- ☐ a standings table worked out from the results, showing played, won, drawn, lost, goals for, goals against, goal difference and points, sorted by points, then goal difference, then goals scored
- ☐ a list of results, newest first, with a button to undo the last one
- ☐ everything survives a refresh, and a "New match night" button clears it all after `confirm()`
- ☐ a Share button that uses the phone's share menu, and copies the text instead where sharing is not available

Nice to have

- the leader's row highlighted
- a "who plays next" suggestion: the two players who have played the fewest games
- the head-to-head record between any two players

Day by day

- | | |
|--------|--|
| Day 57 | Plan the data: a <code>players</code> array of names and a <code>results</code> array of objects. Build the page layout and the add-player form with its checks. |
| Day 58 | The result form: two selects filled from <code>players</code> , two score inputs, the checks, and pushing a result object into <code>results</code> . |
| Day 59 | Write <code>standings(players, results)</code> , which returns one row object per player. Render it, sorted. |
| Day 60 | The results list and undo. Write one <code>render()</code> function that redraws everything, and call it after every change. |
| Day 61 | Save to and load from <code>localStorage</code> . Add New match night with <code>confirm()</code> . |
| Day 62 | Share: build a text summary and pass it to <code>navigator.share()</code> , falling back to <code>navigator.clipboard.writeText()</code> . |
| Day 63 | Play a full night with six players and ten results. Fix the empty states and check it on the smallest phone you can find. |

New ideas this week

Scores come from inputs, so they arrive as text. Convert them with `Number()` before you add them up, or 2 plus 1 becomes 21.

Start each player's standings row like this, then loop through the results and add to the two rows involved in each one:

script.js

```
const rows = players.map(name => ({
  name, played: 0, won: 0, drawn: 0, lost: 0, gf: 0, ga: 0, points: 0
}));
```

Sharing needs the page to be served over <https> or from your editor's preview, so test the Share button on your GitHub Pages link:

script.js

```
async function shareResults(text) {
  if (navigator.share) {
    await navigator.share({ title: "Match night results", text });
  } else {
    await navigator.clipboard.writeText(text);
    alert("Results copied. Paste them into your group chat.");
  }
}
```

Done when

- ☐ a full night with six players works from start to finish with no errors in the console
- ☐ refreshing in the middle of the night loses nothing
- ☐ the shared text reads clearly when pasted into WhatsApp

If you are stuck

If the standings look wrong, write the right table for three players and three results on paper. Then `console.table` your rows and compare them line by line.

JSON is how data is stored and sent between programs. It looks like JavaScript objects, with stricter rules: keys and strings use double quotes, there are no trailing commas, no comments and no functions.

data/games.json

```
[
  {
    "id": "efootball",
    "title": "eFootball",
    "genre": "Football",
    "platforms": ["mobile", "playstation"],
    "maxPlayers": 2
  }
]
```

Your task

Create a **data** folder with three files: **games.json** with twenty games, **players.json** with sixteen players and **tournaments.json** with six tournaments. Give each tournament an id, title, game id, a start date and time in ISO format, format, number of slots, entry fee and prize. Check each file at jsonlint.com, which works on a phone.

Done when

- ☐ All three files pass the JSON check
- ☐ Every item in a file has the same keys
- ☐ Every id is unique within its file

If you are stuck

The most common JSON errors are single quotes, a comma after the last item, and a missing comma between two items.

Day 65 Loading data with fetch

Phone + server

`fetch` loads a file or asks a server for data. It takes time, so it gives you back a promise. `await` waits for the promise without freezing the page, and it only works inside an `async` function.

```
script.js

async function loadGames() {
  const response = await fetch("data/games.json");
  return await response.json();
}

loadGames().then(games => {
  games.forEach(game => grid.append(renderGame(game)));
});
```

`fetch` cannot read files when you open the HTML straight from your phone's storage. It only works through your editor's preview or run button, or on your GitHub Pages site. The same applies on a laptop, which is what Live Server is for.

Your task

Make `games.html` load its games from `games.json` instead of `data.js`, and build the cards with your day 46 function. Do the same for the leaderboard with `players.json`.

Done when

- ☐ Removing the arrays from `data.js` breaks neither page
- ☐ Adding a game to `games.json` shows a new card after a refresh
- ☐ There are no errors in the console

Day 66 Loading and error states

Phone + server

Loading can be slow, and it can fail. Show something while you wait, and something helpful if it goes wrong. `response.ok` is false when the server answered with an error, like a missing file.

```
script.js

async function loadGames() {
  showSkeletons();
  try {
    const response = await fetch("data/games.json");
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const games = await response.json();
    showGames(games);
  } catch (error) {
    showError("We couldn't load the games.");
    console.error(error);
  }
}
```

Your task

While the games load, show six of your skeleton cards from day 31. If loading fails, show a friendly message with a "Try again" button that runs the loading function again. Test the failure by renaming `games.json` for a moment.

Done when

- ☐ The skeletons show while the data loads
- ☐ A missing file shows your message, not a blank page
- ☐ Try again works once the file is back

Push further

To see the skeletons properly, wait one second before loading with `await new Promise(r => setTimeout(r, 1000));`. Remove it afterwards.

Day 67 Talking to a real API

Phone + server

An API is a web address that gives back data instead of a page. GitHub's API is free and needs no key: `https://api.github.com/users/USERNAME` returns a profile as JSON. Without a key, you can only ask it 60 times an hour, so handle the error when you hit the limit.

script.js

```
const response = await fetch("https://api.github.com/users/your-username");

if (response.status === 404) {
  showMessage("No GitHub user with that name.");
} else if (!response.ok) {
  showMessage("GitHub is busy. Try again later.");
} else {
  const user = await response.json();
  // user.avatar_url, user.name, user.bio, user.public_repos, user.html_url
}
```

Your task

Add a "Built by" card to your about page that loads your own GitHub profile: your avatar with alt text, your name, your bio, how many public repositories you have and a link to your profile. It needs internet.

Done when

- ☐ The card shows your real, current profile
- ☐ A wrong username shows a clear message
- ☐ With your internet off, the rest of the page still works

Push further

List your three most recently updated repositories, from `https://api.github.com/users/USERNAME/repos?sort=updated&per_page=3`.

Day 68 Splitting code into modules

[Phone + server](#)

As your code grows, split it into modules. Each file exports what others need and imports what it uses. Nothing leaks out by accident, and module scripts are deferred automatically.

script.js

```
// js/utils.js
export function formatNaira(amount) { ... }

// js/pages/games.js
import { formatNaira } from "../utils.js";
```

index.html

```
<script type="module" src="js/pages/games.js"></script>
```

Import paths need the `.js` on the end. Modules also need a server, the same as `fetch`.

Your task

Reorganise your JavaScript into `js/api.js` for loading data, `js/render.js` for building elements, `js/utils.js` for your helpers, and one file per page in `js/pages/`, each importing only what it needs.

Done when

- ☐ Every page works exactly as it did before
- ☐ Typing one of your variable names into the console says it is not defined, because nothing leaks out of your modules
- ☐ No file is longer than about 150 lines

Day 69 Pages with an id in the address

[Phone + server](#)

One page can show any game by reading an id from its address, like `game.html?id=efootball`.

script.js

```
const id = new URLSearchParams(location.search).get("id");
const game = games.find(g => g.id === id);
```

Link to it from each card with `href="game.html?id=efootball"`.

Your task

Build `game.html`. It shows one game in full: a big image, the description, the platforms, any upcoming tournaments for that game (filter `tournaments.json` by game id) and the players who main it. Make every game card link to its page.

Done when

- ☐ Every card opens the right game's page
- ☐ A missing or wrong id shows "Game not found" with a link back to the library
- ☐ The tab title changes to the game's name

Day 70 Search as you type

Phone + server

Search is a filter that runs on every key press. Make the comparison fair by removing extra spaces and ignoring capitals on both sides.

script.js

```
const clean = text => text.trim().toLowerCase();

search.addEventListener("input", () => {
  const term = clean(search.value);
  const found = games.filter(g =>
    clean(g.title).includes(term) || clean(g.genre).includes(term)
  );
  showGames(found);
});
```

Your task

Add a search box to `games.html` that filters by title, genre and platform as you type. Show "Showing 4 of 20 games" above the grid, and a clear button that empties the search and shows everything again.

Done when

- ☐ " FOOTBALL " finds the same games as "football"
- ☐ The count is always right
- ☐ Clearing brings every game back

Push further

Show a "No games match" message with the search term in it when nothing matches.

Day 71 JavaScript that works for everyone

Laptop recommended

When JavaScript changes the page, screen reader users do not see it change. An `aria-live` region reads out what changed:

```
index.html
```

```
<p id="result-count" aria-live="polite"></p>
```

Anything you can click should be a `button` (it does something) or an `a` (it goes somewhere), because those work with the keyboard for free. To move focus to something that is not normally focusable, give it `tabindex="-1"` and call `.focus()`.

Checking this with a keyboard and a screen reader is much easier on a laptop. On a phone, use TalkBack or VoiceOver and a Bluetooth keyboard.

Your task

Make your search count and your filter results announce themselves with `aria-live`. On match night, after a result is recorded, move focus to a message saying what was recorded. Search your JavaScript for every `addEventListener("click"` and check that each one is on a button or a link.

Done when

- ☐ A screen reader announces the new count after a search
- ☐ After recording a result, focus lands on the confirmation message
- ☐ No click listener is on a `div` or a `span`

Day 72 Speed

Phone

Most of what makes a site slow is images that are too big, and scripts that block the page from showing. PageSpeed Insights (pagespeed.web.dev) tests your live site and tells you what to fix. It works from a phone browser.

Common fixes: compress images, give every image a width and height, lazy load anything below the first screen, and load scripts with `defer` or as modules.

Your task

Test your live site on PageSpeed Insights and look at the mobile results. Write down the score and the top three suggestions. Fix them: compress your images at squoosh.app, check every image has a size, lazy load everything below the first screen and defer every script. Upload the changes and test again.

Done when

- ☐ The before and after scores are written in your changelog
- ☐ Every image is under 200 KB
- ☐ Nothing on the page jumps around as images load

Level 4

The platform

Weekly projects · Days 73 to 100

From here it is boss weeks only. Each one builds a feature too big for an evening: a full game library, tournaments with live brackets, a leaderboard worked out from real match results, and finally a launch.

You already know everything these weeks need, apart from a few new ideas each one introduces. The hard part now is holding a bigger piece of code in your head. Two habits make that much easier. Keep your data in one place and change the page only from one `render` function. And when something breaks, test the smallest piece you can on its own before looking at the whole thing.

By the end of this level

- a searchable, filterable game library with shareable links
- tournaments with registration and knockout brackets
- a leaderboard, player profiles and clan pages built from match results
- a launched site your community actually uses

The game library

Phone + server

The brief

Your games page has grown piece by piece: cards from data, search, saved games, detail pages. This week you join all of it into one proper library with filters and sorting that work together, and addresses you can share. When someone sends `games.html?genre=football&platform=mobile` to the group chat, whoever opens it sees exactly the same list.

Must have

- ☐ one `state` object holding the search term, genre, platforms, sort order and a saved-only switch
- ☐ one `render()` function that filters, sorts and draws the cards from `state`, and nothing else touches the grid
- ☐ a genre `select`, platform checkboxes and the search box from day 70, all working together
- ☐ sorting by name, by max players and by number of upcoming tournaments
- ☐ a "Saved only" switch using your saved games from day 53
- ☐ a "Clear filters" button
- ☐ the current filters kept in the address, and read back when the page loads
- ☐ a result count that screen readers announce, and a helpful message when nothing matches

Nice to have

- filter counts, like "Football (4)", in the genre list
- a "Surprise me" button that opens a random game's page

Day by day

- | | |
|--------|--|
| Day 73 | Write the <code>state</code> object and the <code>render()</code> function. Move your search into it, so the search box only updates <code>state.search</code> and calls <code>render()</code> . |
| Day 74 | Add the genre select and the platform checkboxes. Make them combine with the search. |
| Day 75 | Add sorting. For "most tournaments", count each game's upcoming tournaments from <code>tournaments.json</code> . |
| Day 76 | Add the Saved only switch and the Clear filters button. |
| Day 77 | Keep the state in the address with <code>URLSearchParams</code> and <code>history.replaceState</code> , and read it on page load. |
| Day 78 | Empty state, loading state, error state and the announced result count. |
| Day 79 | Test every combination you can think of. Write the must-have list on paper and tick each item as you prove it works. |

New ideas this week

Every control only changes `state` and calls `render()`. Nothing else draws anything.

script.js

```
const state = { search: "", genre: "all", platforms: [], sort: "name", savedOnly: false };

function render() {
  let list = games.filter(game => matches(game, state));
  list = sortGames(list, state.sort);
  drawCards(list);
  count.textContent = `Showing ${list.length} of ${games.length} games`;
  saveStateToAddress(state);
}

genreSelect.addEventListener("change", () => {
  state.genre = genreSelect.value;
  render();
});
```

To update the address without reloading the page:

script.js

```
const params = new URLSearchParams();
if (state.genre !== "all") params.set("genre", state.genre);
history.replaceState(null, "", `?${params}`);
```

Done when

- ☐ opening a shared filtered link shows the same results as the person who sent it
- ☐ no combination of filters leaves an empty grid without an explanation
- ☐ the grid is only ever drawn by `render()`

If you are stuck

If filters seem to fight each other, you have more than one function drawing the cards. Search your code for every place that changes the grid. There should be exactly one.

Tournaments and brackets

Laptop recommended

The brief

This is the part your community will use most. Tournaments load from your data, show whether they are upcoming, live or finished, let players register, and then run a knockout bracket from the first round to a champion.

Must have

- ☐ a `tournament.html?id=...` page showing the tournament's details, its status worked out from its date, and a countdown if it has not started
- ☐ a registration form asking for gamer tag and platform, with the slots remaining shown, no duplicate tags, and registration closed once it is full or has started
- ☐ registrations saved in `localStorage`, kept separately for each tournament
- ☐ a "Generate bracket" button that shuffles the registered players fairly and pairs them up, giving byes when the number of players is not a power of two
- ☐ tapping a player in a match makes them the winner and moves them into the next round, until one champion is left
- ☐ the bracket drawn as one column per round, scrolling sideways on a phone

Nice to have

- undo a winner
- a printable version of the bracket
- a champion screen with a Share button, reusing your code from day 62

Day by day

Day 80	The tournament page: its details, status from its date, and the countdown from day 54.
Day 81	The registration form, slots remaining, duplicate check and closing rules, saved for each tournament.
Day 82	Write <code>shuffle()</code> and check it gives a different order most times you run it.
Day 83	Build round one with byes. Store the bracket as an array of rounds, where each round is an array of matches like <code>{ a, b, winner }</code> .
Day 84	Tapping a player sets the winner and fills the right spot in the next round.
Day 85	Draw the bracket with CSS grid, one column per round, scrolling sideways on narrow screens.
Day 86	The champion screen, saving the bracket so a refresh keeps it, and a full test with 5, 8 and 11 players.

New ideas this week

A fair shuffle, known as the Fisher-Yates shuffle:

```
script.js

function shuffle(list) {
  const copy = [...list];
  for (let i = copy.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [copy[i], copy[j]] = [copy[j], copy[i]];
  }
  return copy;
}
```

A knockout bracket needs 2, 4, 8, 16 or 32 players. Find the next of those sizes, and the difference is how many byes you need:

```
script.js

let size = 2;
while (size < players.length) size *= 2;
const byes = size - players.length;
```

Give the byes out first, so no match ever has two byes. With five players you need three byes, so round one is three players each against a bye, then the last two players against each other. A player against a bye goes straight through to round two.

Done when

- ☐ the brackets for 5, 8 and 11 players are all correct
- ☐ no match ever has a bye on both sides
- ☐ refreshing the page keeps the bracket exactly as it was

If you are stuck

Draw the bracket for five players on paper first, and write out the array for each round beside it. Make your code produce exactly those arrays in the console before you draw anything on screen.

Players, clans and the real leaderboard

Phone + server

The brief

So far your leaderboard reads wins straight from `players.json`, which is not how real leaderboards work. They are worked out from a list of match results, so they cannot disagree with what happened. This week you build that, along with a profile for every player and a page for every clan.

Must have

- ☐ `matches.json` with at least 40 results, each with a date, game id, two players and two scores
- ☐ a function that works out each player's played, won, drawn, lost, points and form (their last five results as W, D or L) from the matches alone
- ☐ the leaderboard built from those numbers and sortable by tapping any column heading, with `aria-sort` on the sorted column
- ☐ a game filter on the leaderboard
- ☐ `profile.html?tag=...` showing a player's stats, tier, clan, form and last ten matches
- ☐ `clans.json` and `clan.html?id=...` listing each clan's members and their combined points
- ☐ every gamer tag anywhere on the site linking to that player's profile

Nice to have

- a page comparing two players side by side
- an "Edit my profile" form that saves a bio and favourite game to `localStorage` and shows them on your own profile
- an avatar made from the player's initials, in a colour worked out from their tag

Day by day

- | | |
|--------|---|
| Day 87 | Write <code>matches.json</code> . Then write <code>statsFor(tag, matches)</code> and test it against one player whose numbers you work out by hand. |
| Day 88 | Build the leaderboard from the worked-out stats, with sortable headings and <code>aria-sort</code> . |
| Day 89 | The game filter, and the form guide showing the last five results. |
| Day 90 | <code>profile.html</code> with stats and match history. |
| Day 91 | <code>clans.json</code> , the clan pages, and links between players and their clans. |
| Day 92 | One of the nice-to-haves. |
| Day 93 | Make every gamer tag on the site a link to its profile. Test with a tag that does not exist. |

New ideas this week

A colour that is always the same for the same tag, useful for avatars:

script.js

```
function colourFor(tag) {  
  let hash = 0;  
  for (const ch of tag) hash = (hash * 31 + ch.charCodeAt(0)) % 360;  
  return `hsl(${hash} 60% 45%)`;  
}
```

For sortable headings, put a `button` inside each `th`. When one is tapped, set `aria-sort="ascending"` or `"descending"` on its `th` and remove `aria-sort` from the others.

Done when

- ☐ every number on the leaderboard can be checked against `matches.json`
- ☐ every gamer tag on the site is a link
- ☐ an unknown tag shows "Player not found" with a link back to the leaderboard

If you are stuck

If a player's numbers look wrong, filter the matches down to just that player and `console.table` them. Then count their wins with your finger.

Launch week

Laptop recommended

The brief

Your last week is not about new features. It is about making what you have solid enough to hand to your community. Everything here is checking, fixing and finishing, and it is the difference between a project and something people use.

Must have

- ☐ an accessibility pass: every page used with the keyboard only, alt text checked, contrast checked in both themes, headings in order, focus always visible, and [aria-live](#) wherever content changes
- ☐ a speed pass, with the PageSpeed Insights mobile score written down before and after
- ☐ link previews, so your link shows a title, description and picture when it is shared on WhatsApp
- ☐ a [README.md](#) on GitHub saying what Lobby is, with screenshots, how to run it and what you would build next
- ☐ Eruda and every debugging [console.log](#) removed
- ☐ a final version live and tested on at least three different phones

Nice to have

- a web app manifest, so people can add Lobby to their home screen like an app
- a "What I learned" page on the site

Day by day

Day 94	Accessibility pass: use every page with the keyboard only and fix everything you find.
Day 95	Speed pass with PageSpeed Insights, then fix the top suggestions.
Day 96	Responsive pass on real devices. Borrow two phones and a laptop.
Day 97	Link preview tags and a 1200 by 630 pixel share image. Test by sending the link to yourself on WhatsApp.
Day 98	Clean up: delete unused files and code, make names consistent, add comments wherever the code is not obvious. Write the README.
Day 99	Deploy the final version. Ask three people to try to break it and write down everything that goes wrong.
Day 100	Fix what they found. Then post the link in your community with a short message about what it does.

New ideas this week

Link previews come from Open Graph tags in each page's **head**:

index.html

```
<meta property="og:title" content="Lobby | Gaming community">
<meta property="og:description" content="Tournaments, match nights and leaderboards for our crew.">
<meta property="og:image" content="https://your-username.github.io/lobby/images/share.png">
<meta property="og:url" content="https://your-username.github.io/lobby/">
<meta property="og:type" content="website">
```

The image address must be a full <https> address, not a relative path. WhatsApp remembers previews, so if you change the image and the old one still shows, test with a slightly different link, such as your address with [?v=2](#) on the end.

Done when

- ☐ sharing the link on WhatsApp shows your picture and description
- ☐ no page has a single error in the console
- ☐ three people have used Lobby on their own phones
- ☐ the README on GitHub has at least two screenshots

After day 100

You now have a working community site built from nothing but HTML, CSS and JavaScript, and you understand every line of it. These are the next steps, roughly in order.

A server and a database

Lobby's biggest limit is that everything people save stays on their own phone. Real accounts, shared tournament registrations and a chat everyone can read need a server and a database. The two usual routes are to learn Node.js and write your own server, or to use a service such as Firebase or Supabase that gives you accounts and a database without running a server yourself. Either way, your Lobby front end stays almost exactly as it is.

Git, properly

If you have been uploading through the GitHub website, learn Git on a laptop: branches, commits with good messages, and going back to an earlier version when something breaks.

A framework, later

React, Vue and Svelte are popular because they make the `state` and `render()` pattern from boss week 4 easier on large sites. You already understand the problem they solve, which makes them much easier to learn. Do not start one until plain JavaScript feels comfortable.

Keep building

The fastest way to improve now is to build a second, different site from an empty folder, without this guide open. A booking page for a barber, a menu for a local restaurant, or a results site for your school's football league. Pick something a real person would use, and ask them what they need before you start.

Progress tracker

Tick a box when you finish a day's task, not when you read it. Shaded boxes are boss week days. Missing a day is fine. Just pick up where you stopped.

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100